สรุปโค้ด Client – Server

Multi-threaded File Download

เปรียบเทียบ Traditional I/O กับ NIO Native Transfer

ไฟล์ที่เกี่ยวข้อง: Server.java · Client.java · MakeFile.java · HOW TO RUN.txt · การวิเคราะห์ผลการทดลอง.md

วิชา OS (Week 09 I/O Lab) · ภาษา Java

สารบัญ

1. ภาพรวมโปรเจกต์และเป้าหมาย
2. ศัพท์พื้นฐานที่ต้องรู้
3. สถาปัตยกรรมและ Protocol
4. Server.java อย่างละเอียด
5. Client.java อย่างละเอียด
6. MakeFile.java และวิธีรัน
7. ลำดับการทำงานตั้งแต่ต้นจนจบ
8. การเปรียบเทียบ Traditional กับ NIO
9. ผลการทดลองและการวิเคราะห์
10. เทคนิคและจุดออกแบบที่สำคัญ
11. คำถามที่อาจถูกถามตอน Demo
12. จับคู่ข้อกำหนดของโจทย์กับตำแหน่งในโค้ด

1. ภาพรวมโปรเจกต์และเป้าหมาย

โปรเจกต์นี้คือระบบ **โหลดไฟล์ขนาดใหญ่ผ่านเครือข่าย (TCP)** โดย Client แบ่งไฟล์เป็นช่วง ๆ แล้วให้ **worker thread หลายตัว** ขอข้อมูลคนละช่วงจาก Server พร้อมกัน จากนั้นแต่ละ worker เขียนข้อมูลลงไฟล์ปลายทางตรงตำแหน่งของตัวเอง โดยไม่ต้องนำชิ้นส่วนมารวมกันอีกครั้ง

เป้าหมายที่ต้องการวัดมี 2 ข้อ

- ใช้ 1 worker เทียบกับ 10 workers แล้วเร็วขึ้นหรือไม่
- การส่งข้อมูลแบบ **Traditional I/O** (ผ่าน `byte[]`) เทียบกับ **NIO** (`transferTo / transferFrom`) แบบไหนเร็วกว่ากัน

> ทุกรอบทดสอบมีการตรวจ **ขนาดไฟล์** และ **SHA-256** เทียบกับต้นฉบับบน Server จึงมั่นใจได้ว่าข้อมูลถูกต้อง สิ่งที่แตกต่างระหว่างการทดลองคือประสิทธิภาพเท่านั้น

ไฟล์ในโปรเจกต์

ไฟล์	หน้าที่
`Server.java`	รับคำสั่งจาก Client ตอบข้อมูลไฟล์ และส่งข้อมูลดิบเป็นช่วง ๆ ด้วย thread pool 20 ตัว
`Client.java`	ส่งคำสั่ง, แบ่งงานให้ worker, รับข้อมูลลงไฟล์, ตรวจความถูกต้อง, จับเวลา, บันทึก log
`MakeFile.java`	สร้างไฟล์ทดสอบที่เป็นข้อมูลสุ่ม (ไม่ได้ใช้ในการดาวน์โหลดโดยตรง)
`HOW TO RUN.txt`	ขั้นตอน compile และตัวอย่างคำสั่งรัน
`การวิเคราะห์ผลการทดลอง.md`	ตารางผล 3 รอบ และคำอธิบายว่าทำไมเป็นเช่นนั้น

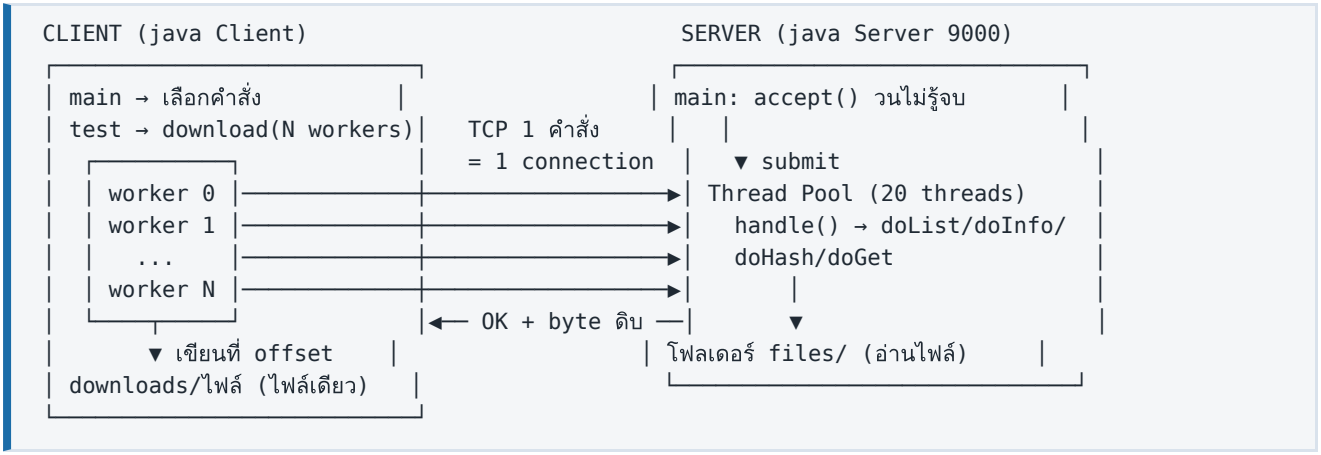
หมายเหตุ: PDF โจทย์ "Job Scheduler" ที่แนบมาด้วยเป็นคนละงานกับโปรเจกต์นี้ จึงไม่รวมในเอกสารฉบับนี้

2. ศัพท์พื้นฐานที่ต้องรู้

คำศัพท์	ความหมาย
Server / Client	ฝั่งให้บริการ / ฝั่งขอใช้บริการ
Port	เลขช่องของเครื่อง (เช่น 9000) ทำให้หลายโปรแกรมใช้เครือข่ายพร้อมกันได้โดยไม่ปนกัน
localhost	ชื่อที่หมายถึงเครื่องตัวเอง ใช้ทดสอบ Server และ Client ในเครื่องเดียว
Socket / SocketChannel	ท่อสื่อสารสองทางระหว่างสองโปรแกรม ฝั่งหนึ่งเขียน อีกฝั่งอ่าน
Protocol	กติกาการคุยกัน เช่น Client ส่ง `INFO a.bin` แล้ว Server ต้องตอบ `SIZE ...`
Offset / Length	เริ่มอ่านที่ byte ที่เท่าไร / อ่านกี่ byte (offset=100, length=50 คือ byte ที่ 100 ถึง 149)
Thread / Thread Pool	เส้นทางการทำงานหนึ่งเส้น / กลุ่ม thread ที่สร้างไว้ล่วงหน้าเพื่อรับงานหมุนเวียน
Buffer	พื้นที่พักข้อมูลชั่วคราว (ในโค้ดใช้ 64 KB) อ่าน/เขียนทีละก้อนเร็วกว่าทีละ byte
SHA-256	ค่า hash ลายนิ้วมือของไฟล์ ข้อมูลต่างกันแม้ byte เดียวค่าจะเปลี่ยนทั้งหมด
Page Cache	หน่วยความจำที่ระบบปฏิบัติการใช้เก็บไฟล์ที่เพิ่งอ่านไว้ ทำให้อ่านซ้ำเร็วมาก
Zero-copy	การย้ายข้อมูลโดยให้ OS ทำตรง ๆ ลดการคัดลอกผ่านหน่วยความจำของโปรแกรม
Loopback	เครือข่ายเสมือนภายในเครื่อง (localhost) ไม่ผ่านการ์ดเครือข่ายจริง

3. สถาปัตยกรรมและ Protocol

3.1 โครงสร้างระบบ



3.2 กติกา Protocol (ข้อความเป็นบรรทัด ลงท้ายด้วย `\n`)

Client ส่ง	Server ตอบ	ความหมาย
`LIST`	`FILE <ชื่อ> <ขนาด>` ซ้ำหลายบรรทัด แล้ว `END`	แสดงรายการไฟล์
`INFO <file>`	`SIZE <bytes>`	ขนาดไฟล์
`HASH <file>`	`HASH <sha256>`	ลายนิ้วมือไฟล์ต้นฉบับ
`GET <file> <offset> <length> [mode]`	`OK <length>` ตามด้วย **byte ดิบ** ตามจำนวน length	ขอข้อมูลเป็นช่วง mode = traditional หรือ nio

รหัส error	เมื่อไรที่เกิด
`ERROR 400`	คำสั่งผิดรูปแบบ, จำนวนคำไม่ถูก, mode ไม่รู้จัก, offset/length ไม่ใช่ตัวเลข
`ERROR 404`	ไม่พบไฟล์ (หรือชื่อไฟล์ไม่ปลอดภัย)
`ERROR 416`	ขอช่วงที่เกินขนาดไฟล์

หลักสำคัญ: 1 connection = 1 คำสั่ง แล้วปิด ส่วนตอบ `GET` จะมีบรรทัดข้อความ `OK n` ก่อน แล้วจึงตามด้วยข้อมูลไบนารี n byte ทันที ทำให้ Client ต้องอ่านบรรทัดแรกแบบไม่ล้ำไปกินข้อมูลไฟล์

4. Server.java อย่างละเอียด

4.1 ค่าคงที่

ตัวแปร	ความหมาย
`FILES = Path.of("files")`	โฟลเดอร์ที่ Server แชร์ไฟล์ (อ้างอิงจากที่ที่สั่งรัน)
`BUFFER = 64 * 1024`	ขนาดกล่องพักข้อมูล 64 KB ใช้ใน traditional และตอนคำนวณ hash

4.2 main(): เปิดรับ connection

```
ExecutorService pool = Executors.newFixedThreadPool(20);
try (ServerSocketChannel server = ServerSocketChannel.open()) {
    server.bind(new InetSocketAddress(port));
    while (true) {
        SocketChannel client = server.accept();   // หยุดรอจนมี Client
        pool.submit(() -> handle(client));        // ส่งต่อให้ thread ใน pool
    }
}
```

- **Thread pool 20 ตัว:** รับได้พร้อมกันสูงสุด 20 connection ที่เกินจะรอคิว
- **accept()** บล็อกจนมีผู้เชื่อมต่อ แล้ว main thread กลับไปรอคนต่อไปทันที ไม่ได้ทำงานหนักเอง
- **ใช้ ServerSocketChannel** เพราะโหมด NIO ต้อง `transferTo()` ลง `SocketChannel` โดยตรง

4.3 handle(): ดูแล connection เดียว

1. เปิดตัวอ่านบรรทัด (`BufferedReader` ห่อ channel) และตัวเขียน (`OutputStream`) ด้วย try-with-resources เพื่อให้ปิด connection อัตโนมัติ
2. `readLine()` อ่านคำสั่ง 1 บรรทัด ถ้าได้ `null` (Client ตัดสาย) ให้จบทันที
3. `trim().split("\\s+")` แยกคำสั่งเป็นคำ ๆ แล้วเปลี่ยนคำสั่งเป็นตัวพิมพ์ใหญ่
4. เลือกเมธอดตามคำสั่ง ไม่ตรงกับอะไรให้ตอบ `ERROR 400 bad request`
5. `catch (Exception)` พิมพ์ error แต่ไม่ทำให้ Server ล่ม thread กลับไปรับงานต่อได้

4.4 เมธอดของแต่ละคำสั่ง

เมธอด	การทำงาน
`doList`	วนดูทุกรายการในโฟลเดอร์ ส่ง `FILE ชื่อ ขนาด` เฉพาะไฟล์จริง แล้วปิดด้วย `END`
`doInfo`	หาไฟล์ด้วย `findFile` ไม่เจอ → 404 เจอ → `SIZE ขนาด`
`doHash`	เหมือน doInfo แต่คำนวณ SHA-256 แล้วตอบ `HASH ค่า`
`doGet`	ตรวจหลายด่าน แล้วส่งข้อมูลช่วงที่ขอ (หัวข้อ 4.5)

4.5 doGet(): ด่านตรวจตามลำดับ

#	การตรวจ	ถ้าไม่ผ่าน
1	จำนวนคำต้อง 4 หรือ 5	ERROR 400
2	mode ต้องเป็น traditional หรือ nio (ไม่ใส่ = traditional)	ERROR 400
3	offset, length แปลงเป็นตัวเลขได้	ERROR 400
4	ไฟล์ต้องมีอยู่และชื่อปลอดภัย	ERROR 404
5	`offset < 0 || length < 0 || offset > size || length > size - offset`	ERROR 416

ทำไมเขียน `length > size - offset` ไม่ใช้ `offset + length > size`
ถ้า offset และ length มีค่ามหาศาล การบวกกันอาจ **overflow** จนเป็นค่าติดลบและหลุดผ่านการตรวจโดยไม่ตั้งใจ การลบ (เมื่อรู้ว่า offset ≤ size แล้ว) ปลอดภัยกว่า

เมื่อผ่านทุกด่าน Server ส่ง `OK <length>` ก่อน แล้วเรียก `sendNio` หรือ `sendTraditional` ตาม mode

4.6 sendTraditional(): ผ่าน user space

```
raf.seek(offset);
long left = length;
while (left > 0) {
    int n = raf.read(buffer, 0, (int) Math.min(buffer.length, left));
    if (n == -1) throw new IOException("file ended early");
    out.write(buffer, 0, n);
    left -= n;
}
out.flush();
```

- `RandomAccessFile` + `seek` กระโดดไปตำแหน่ง offset ได้ทันที
- อ่านทีละไม่เกิน 64 KB และไม่เกิน `left` เพื่อไม่ส่งเกินช่วงที่ขอ
- `n == -1` หมายถึงไฟล์จบก่อนครบ จึงโยน error
- เส้นทางข้อมูล: ดิสก์/Page Cache → `byte[]` ของโปรแกรม → socket

4.7 sendNio(): ให้ OS ย้ายข้อมูลเอง

```
try (FileChannel source = FileChannel.open(file, StandardOpenOption.READ)) {
    long sent = 0;
    while (sent < length) {
        long n = source.transferTo(offset + sent, length - sent, client);
        if (n <= 0) throw new IOException("transferTo made no progress");
        sent += n;
    }
}
```

- `transferTo` อาจย้ายได้น้อยกว่าที่ขอ จึงต้องวนลูป และตำแหน่งเริ่มรอบถัดไปคือ `offset + sent`
- `n <= 0` ถือว่าไม่คืบหน้า โยน error เพื่อกันลูปค้างไม่รู้จบ
- เส้นทางข้อมูล: Page Cache → socket โดยตรง (zero-copy เมื่อระบบรองรับ)

4.8 เมธอดช่วย

เมธอด	รายละเอียด
`sha256`	อ่านไฟล์ทีละ 64 KB ป้อนให้ `MessageDigest` (ไม่โหลดทั้งไฟล์เข้าหน่วยความจำ) แล้วแปลงผลเป็นเลขฐาน 16 ยาว 64 ตัวด้วย `%064x`
`findFile`	ตรวจว่า `Path.of(name).getFileName()` เท่ากับ name เดิมหรือไม่ เพื่อกัน path traversal เช่น `..\secret.txt` และจับ `InvalidPathException` สำหรับชื่อที่มีอักขระต้องห้ามบน Windows
`send`	พิมพ์ log บนจอ Server แล้วเขียนข้อความ + `\n` ลง socket และ flush ใช้ `\n` ตรง ๆ ไม่ใช้ `println` เพราะ Windows จะใส่ `\r\n` ทำให้ protocol เพี้ยน

5. Client.java อย่างละเอียด

5.1 ตัวแปรระดับคลาส

ตัวแปร	ความหมาย
`HOST = "localhost"`	เชื่อมต่อเครื่องตัวเอง
`BUFFER = 64 * 1024`	กล่องพัก 64 KB
`port`	ไม่ใช่ final เพราะรู้ค่าตอนรัน (รับจาก command line)
`report` (StringBuilder)	สะสมข้อความผลทดสอบทั้งหมด เพื่อเขียนลงไฟล์ log ทีเดียวตอนจบ

5.2 main(): แยกคำสั่ง

พิมพ์	เรียกเมธอด
`Client 9000 LIST`	`list()`
`Client 9000 INFO <file>`	`request("INFO ...")` แล้วพิมพ์คำตอบ
`Client 9000 HASH <file>`	`request("HASH ...")` แล้วพิมพ์คำตอบ
`Client 9000 GET <file> <off> <len> [mode]`	`getRange(...)` หลังผ่าน `isValidGet`
`Client 9000 <file> <mode> <1|3>`	`test(...)` โหมดทดสอบความเร็ว

- port ต้องเป็นตัวเลข 1–5 หลัก (regex `\d{1,5}`)
- จำนวนรอบทดสอบรับได้เฉพาะ 1 หรือ 3 ตามข้อกำหนด "อย่างน้อย 3 ครั้ง"
- ถ้าเกิด `IOException` (เช่น ERROR 404, เชื่อมต่อไม่ได้) พิมพ์ข้อความแล้ว `System.exit(1)`

5.3 ฟังก์ชันสื่อสารพื้นฐาน

เมธอด	การทำงาน
`request(line)`	เปิด socket, ส่ง 1 บรรทัด, อ่านตอบ 1 บรรทัด (ใช้กับ INFO และ HASH)
`list()`	ส่ง LIST แล้วอ่านทีละบรรทัดพิมพ์ออกจอจนเจอ `END` หรือ `ERROR`
`expect(reply, prefix)`	ตรวจว่าคำตอบขึ้นต้นตามคาด ถ้าไม่ใช่ (เช่น ERROR ...) โยน `IOException` พร้อมข้อความของ Server
`getSize / getHash`	เรียก request แล้ว expect เพื่อตัดคำนำหน้า `SIZE` / `HASH`
`readLine(in)`	อ่านทีละ **1 byte** จนเจอ `\n`
`sendLine(out, text)`	ต่อ `\n` แปลงเป็น UTF-8 เขียนแล้ว flush

ทำไม readLine ต้องอ่านทีละ byte ไม่ใช้ BufferedReader?
`BufferedReader` อ่านล่วงหน้าเป็นก้อนใหญ่ จะดูดข้อมูลไฟล์ที่ตามหลังบรรทัด `OK n` ไปไว้ใน buffer ของมัน ทำให้ส่วนที่รับต่อขาดหาย การอ่านทีละ byte หยุดตรง `\n` พอดี byte ที่เหลือยังอยู่ใน socket ครบ

5.4 fetchRange(): รับข้อมูล 1 ช่วง

ใช้ร่วมกันทั้งคำสั่ง GET และ worker ในโหมดทดสอบ ประกอบคำขอ `GET name offset length mode` แล้วเรียก `fetchNio` หรือ `fetchTraditional` โดยให้ข้อมูลไปเขียนในไฟล์ปลายทางที่ตำแหน่ง `writeAt`

5.5 fetchTraditional()

```
try (Socket s = new Socket(HOST, port);
     RandomAccessFile file = new RandomAccessFile(target.toFile(), "rw")) {
    sendLine(s.getOutputStream(), request);
    InputStream in = s.getInputStream();
    expect(readLine(in), "OK ");
    file.seek(writeAt);
    byte[] buffer = new byte[BUFFER];
    long left = length;
    while (left > 0) {
        int n = in.read(buffer, 0, (int) Math.min(buffer.length, left));
        if (n == -1) throw new IOException("connection closed early");
        file.write(buffer, 0, n);
        left -= n;
    }
}
```

- แต่ละ worker เปิด socket และ `RandomAccessFile` ของ **ตัวเอง** จึงไม่แย่งตัวชี้ตำแหน่ง (file pointer) กัน
- หลังอ่าน header ให้ `seek(writeAt)` แล้วรับข้อมูลลงไฟล์ตามลำดับ
- `n == -1` คือ Server ตัดการเชื่อมต่อก่อนส่งครบ

5.6 fetchNio()

```
try (SocketChannel socket = SocketChannel.open(new InetSocketAddress(HOST, port));
     FileChannel file = FileChannel.open(target, StandardOpenOption.WRITE)) {
    sendLine(Channels.newOutputStream(socket), request);
    expect(readLine(Channels.newInputStream(socket)), "OK ");
    long done = 0;
    while (done < length) {
        long n = file.transferFrom(socket, writeAt + done, length - done);
        if (n <= 0) throw new IOException("connection closed early");
        done += n;
    }
}
```

- ส่งคำสั่งและอ่าน header ผ่านสตรีมที่ห่อ channel ไว้ (อ่านทีละ byte จึงไม่ล้ำเข้าข้อมูลไฟล์)
- `transferFrom(socket, ตำแหน่งเขียน, จำนวน)` ให้ OS ย้ายข้อมูลจาก socket ลงไฟล์โดยตรง ตำแหน่งเขียนจึงเป็น `writeAt + done`

5.7 download(): โหลดทั้งไฟล์ด้วย N workers

1. `getSize` แล้ว `createFile` สร้างไฟล์ปลายทางเปล่าขนาดเท่าต้นฉบับ (ลบของเก่าก่อน)
2. คำนวณ `part = size / workers` แล้วสร้าง thread pool ขนาดเท่าจำนวน worker

3. เริ่มจับเวลาด้วย `System.nanoTime()`
4. `submit` งานละช่วง: worker ที่ i เริ่มที่ `i * part` ส่วนคนสุดท้ายรับ `size - offset` เพื่อรวมเศษที่หารไม่ลงตัว
5. `t.get()` รอทุก worker เสร็จ ถ้ามีตัวใดพังจะได้ `ExecutionException` แล้วแกะสาเหตุออกมาแจ้ง
6. `finally { pool.shutdown(); }` ปิด thread เสมอ ไม่ให้โปรแกรมค้าง
7. หยุดจับเวลา แล้วตรวจ **ขนาดไฟล์** และ **SHA-256** (ไม่นับเวลาตรวจ)

ตัวอย่าง: ไฟล์ 1000 MB ใช้ 10 workers	offset	length
Worker 0	0	100 MB
Worker 1	100 MB	100 MB
...	...	...
Worker 9 (คนสุดท้าย)	900 MB	size − offset (รวมเศษ)

> **จุดฉลาดของการออกแบบ:** ทุก worker เขียนลงไฟล์ปลายทางไฟล์เดียวกันที่ offset ของตนเอง ช่วงไม่ทับกัน จึง **ไม่ต้อง merge** และไม่ต้องใช้ lock

รายละเอียดทางเทคนิค: `return null;` ใน lambda ทำให้ Java มองเป็น `Callable` ซึ่งโยน `IOException` ได้ ถ้าไม่ return จะกลายเป็น `Runnable` ที่โยน checked exception ไม่ได้

5.8 test(): โหมดทดสอบความเร็ว

1. ถามขนาดและ SHA-256 ของไฟล์ต้นฉบับจาก Server
2. **Warm-up 1 รอบ (ไม่นับ)** เพื่อให้ไฟล์เข้า Page Cache และให้ JVM อุ่นเครื่อง ผลจะไม่เพี้ยนจากรอบแรกที่ช้ากว่าปกติ
3. วนทดสอบ 1 worker แล้ว 10 workers ตามจำนวนรอบที่สั่ง บันทึกเวลาและ MB/s ด้วย `say()`
4. ถ้ามีหลายรอบ คำนวณค่าเฉลี่ยของแต่ละกรณี
5. เขียนรายงานลงไฟล์ `logs/<mode>-<runs>run-<เวลา>.txt`

`say()` พิมพ์ขึ้นจอและเก็บสำเนาลง `report` พร้อมกัน

5.9 เมธอดช่วยอื่น ๆ

เมธอด	การทำงาน
`getRange`	โหลดช่วงเดียวเป็นไฟล์ `downloads/ชื่อ.range-start-end` ถ้า Server ตอบ error ลบไฟล์ทิ้ง แล้วโชว์ hex 64 byte แรก
`createFile`	สร้างโฟลเดอร์, ลบไฟล์เก่า (กันข้อมูลรอบก่อนค้างจนตรวจไม่พบว่าโหลดผิด), กำหนดขนาดไฟล์ล่วงหน้าด้วย `setLength`
`printHex`	แสดง 64 byte แรก บรรทัดละ 16 byte โดยตัวเลขซ้ายคือตำแหน่งในไฟล์ต้นฉบับ (`%08X`)
`sha256`	เหมือนฝั่ง Server เพื่อคำนวณ hash ของไฟล์ที่โหลดมาเทียบกัน
`isValidGet` / `isMode`	ตรวจรูปแบบคำสั่งฝั่ง Client ก่อนส่ง (offset/length เป็นตัวเลข 1–18 หลัก, mode ถูกต้อง)

6. MakeFile.java และวิธีรัน

6.1 MakeFile.java

- รับ 2 ค่า: ที่อยู่ไฟล์ และขนาดเป็น MB
- ใช้ `new Random(42)` seed คงที่ จึงได้ไฟล์เดิมทุกครั้ง
- เขียนทีละ 1 MB และ **สุ่ม byte ใหม่ทุกก้อน** เพื่อให้ไม่มีส่วนใดซ้ำกัน ถ้า worker เขียนผิดตำแหน่ง hash จะไม่ตรงและตรวจพบทันที

6.2 ขั้นตอนรัน

```
# 1. Compile
javac -encoding UTF-8 Server.java Client.java MakeFile.java

# 2. สร้างไฟล์ทดสอบ (เก็บในโฟลเดอร์ files)
java MakeFile files\bigfile.bin 100

# 3. รัน Server (Terminal 1)
java Server 9000

# 4. รัน Client (Terminal 2)
java Client 9000 LIST
java Client 9000 INFO bigfile.bin
java Client 9000 HASH bigfile.bin
java Client 9000 GET bigfile.bin 0 1000 nio
java Client 9000 INFO nofile.bin                  # ทดสอบ ERROR 404
java Client 9000 GET bigfile.bin 999999999 10     # ทดสอบ ERROR 416
java Client 9000 bigfile.bin traditional 3        # ทดสอบความเร็ว
java Client 9000 bigfile.bin nio 3
```

โฟลเดอร์ `downloads` เก็บไฟล์ที่โหลดมา และ `logs` เก็บบันทึกผลทดสอบ

7. ลำดับการทำงานตั้งแต่ต้นจนจบ

ตัวอย่างคำสั่ง `java Client 9000 bigfile.bin nio 3`

```
Client                                          Server
  │── INFO bigfile.bin ──────────────────────────►│
  │◄──────────────────────────── SIZE 1048576000 ─│
  │── HASH bigfile.bin ──────────────────────────►│  (คำนวณ SHA-256)
  │◄──────────────────────────── HASH a3f1...   ──│
  │
  │  Warm-up: download 10 workers (ไม่นับผล)
  │
  │  รอบที่ 1: 1 worker
  │── GET bigfile.bin 0 1048576000 nio ──────────►│
  │◄═══════════ byte ดิบ (transferTo/From) ═══════│
  │  ตรวจ size + SHA-256 → บันทึกเวลา
  │
  │  รอบที่ 1: 10 workers (10 connection พร้อมกัน)
  │── GET ... 0 100MB ──────────► thread 1
  │── GET ... 100MB 100MB ──────► thread 2
  │      ...                        ...
  │── GET ... 900MB (เศษ) ──────► thread 10
  │  ตรวจ size + SHA-256 → บันทึกเวลา
  │
  │  ทำซ้ำรอบ 2, 3 → เฉลี่ย → เขียนไฟล์ logs/
```

8. การเปรียบเทียบ Traditional กับ NIO

	Traditional I/O	NIO Native Transfer
ฝั่ง Server	`RandomAccessFile.read → byte[] → out.write`	`FileChannel.transferTo(..., socketChannel)`
ฝั่ง Client	`in.read → byte[] → file.write`	`FileChannel.transferFrom(socketChannel, ...)`
เส้นทางข้อมูล	ผ่านหน่วยความจำของโปรแกรม (user space)	OS ย้ายให้เอง ลดการคัดลอก
จำนวนการคัดลอก	มากกว่า	น้อยกว่า (zero-copy เมื่อรองรับ)
ความซับซ้อนของโค้ด	ตรงไปตรงมา	สั้นกว่า แต่ต้องวนลูปเมื่อย้ายได้ไม่ครบ
ข้อควรระวัง	ต้องคุมขนาด buffer และ `left` เอง	ต้องเช็ก `n <= 0` กันลูปค้าง

9. ผลการทดลองและการวิเคราะห์

ไฟล์ 1000 MB · Windows 11 · Java 23.0.1 · ค่าเฉลี่ย 3 รอบ · Client และ Server อยู่เครื่องเดียวกัน

วิธี	1 worker	10 workers	Speedup
Traditional	3.53 s (283.6 MB/s)	8.12 s (123.2 MB/s)	0.43 เท่า (ช้าลง)
NIO	3.18 s (314.5 MB/s)	2.96 s (337.8 MB/s)	1.07 เท่า

ทำไม 10 workers จึงไม่เร็วขึ้น 10 เท่า

1. **ทรัพยากรร่วมไม่ได้แบ่งตามจำนวน worker:** ทุก worker ใช้ CPU, memory bandwidth, loopback และดิสก์ชุดเดียวกัน เมื่อคอขวดอยู่ที่ทรัพยากรร่วม การเพิ่ม thread จึงไม่เพิ่มความเร็ว
2. **Client และ Server อยู่เครื่องเดียวกัน:** แย่ง CPU กันเอง และมี thread รวมสูงสุดราว 30 ตัว ทำให้เกิด context switch มากขึ้น
3. **เขียนไฟล์ปลายทาง 10 ตำแหน่งพร้อมกัน:** ดิสก์ต้องเขียนกระจาย และ OS ต้องจัดการ dirty page ของไฟล์ 1000 MB
4. **ต้นทุนของการทำงานหลาย thread:** สร้าง connection 10 ครั้ง, จัดตาราง thread และ buffer ของแต่ละ worker ซึ่งเกินประโยชน์ที่ได้เมื่อทรัพยากรหลักอิ่มตัวแล้ว

ทำไม NIO จึงไม่ชนะทุกครั้ง

- ที่ 1 worker, Traditional เร็วกว่าชัดเจนในสองรอบแรก ส่วนที่ 10 workers NIO ชนะทั้งค่าเฉลี่ยและทุกรอบ
- บน loopback ข้อมูลไม่ผ่านการ์ดเครือข่ายจริง และ Traditional ที่ใช้ buffer ก็มีประสิทธิภาพสูงอยู่แล้ว ส่วนต่างจึงเล็ก
- รอบที่ช้าผิดปกติอาจมาจาก Page Cache ถูกนำออก หรือการ flush ข้อมูลลงดิสก์เบื้องหลัง

> **ข้อสรุป:** ไม่มีวิธีใดชนะในทุกเงื่อนไข ผลขึ้นกับจำนวน worker, สภาพของ cache และภาระของเครื่องขณะทดสอบ

ข้อจำกัดของการทดสอบผ่าน localhost

ปัจจัย	ผลกระทบ
Page Cache	Server อ่านจาก RAM เป็นหลัก ตัวเลขสะท้อนความเร็วหน่วยความจำและ CPU มากกว่าดิสก์จริง อาจสูงกว่าสภาพใช้งานจริง
Loopback network	ไม่มีความหน่วงหรือแบนด์วิดท์จำกัดแบบเครือข่ายจริง ผลของการเพิ่ม worker จึงไม่เหมือนจริง
Storage cache	SSD และ OS ใช้ write-back เวลาเขียนที่วัดอาจยังไม่รวมเวลาเขียนลงสื่อจริง และอาจกระทบรอบถัดไป
แชร์ CPU/หน่วยความจำ	ในสภาพจริงสองฝั่งอยู่คนละเครื่องจึงไม่แย่ง CPU กัน

10. เทคนิคและจุดออกแบบที่สำคัญ

เทคนิค	เหตุผล
try-with-resources	ปิด socket/ไฟล์อัตโนมัติ แม้เกิด error ป้องกัน connection และไฟล์ค้าง
ลูปจนครบจำนวน (`left`, `sent`, `done`)	การอ่าน/เขียนครั้งเดียวไม่รับประกันว่าจะได้ครบ
ตรวจ `-1` และ `n <= 0`	กันลูปไม่รู้จบเมื่ออีกฝั่งตัดการเชื่อมต่อ
ใช้ `\n` ท้ายบรรทัดเสมอ	ให้สองฝั่งตกลงตรงกันว่าบรรทัดจบที่ไหน ไม่ขึ้นกับ OS
`flush()` หลังเขียน	บังคับให้ข้อมูลออกจาก buffer ไม่ค้างรอ
เขียนที่ offset ของตัวเอง	ไม่ต้อง merge และไม่ต้อง lock
สร้างไฟล์ใหม่ทุกรอบ	กันข้อมูลรอบก่อนค้างจนตรวจไม่พบว่าโหลดผิด
ข้อมูลทดสอบสุ่มทั้งไฟล์	ถ้าเขียนผิดตำแหน่ง hash จะไม่ตรง
Warm-up 1 รอบไม่นับผล	ลดผลของ Page Cache เย็นและ JVM ที่ยังไม่อุ่น
ตรวจ size + SHA-256 นอกช่วงจับเวลา	เวลาที่วัดคือเวลาโหลดล้วน ๆ แต่ยังยืนยันความถูกต้อง
`findFile` กัน path traversal	ไม่ให้ Client ขอไฟล์นอกโฟลเดอร์ `files`
`pool.shutdown()` ใน finally	ไม่ให้ thread ค้างจนโปรแกรมไม่จบ

11. คำถามที่อาจถูกถามตอน Demo

คำถาม	แนวคำตอบ
1 connection ทำอะไรได้บ้าง	รับ 1 คำสั่ง ตอบ แล้วปิด ส่วน GET ตอบ `OK n` ตามด้วยข้อมูลดิบ n byte
ทำไมต้องส่ง `OK length` ก่อน	เพื่อให้ Client รู้ว่าข้อมูลดิบจะมีกี่ byte และแยก error (`ERROR ...`) ออกจากข้อมูล
worker หลายตัวเขียนไฟล์เดียวกันไม่ชนกันหรือ	ไม่ชน เพราะแต่ละตัวเปิด handle ของตัวเอง และเขียนคนละช่วง offset ที่ไม่ทับกัน
ทำไมไม่ใช้ BufferedReader ฝั่ง Client	เพราะจะอ่านล่วงหน้ากินข้อมูลไฟล์ที่ตามหลัง header
ช่วงสุดท้ายทำไมใช้ `size - offset`	เพื่อรับเศษที่หารไม่ลงตัวจาก `size / workers`
ทำไมตรวจช่วงด้วย `length > size - offset`	กัน integer overflow ที่เกิดจาก `offset + length`
ทำไมต้องลูปใน `transferTo/From`	แต่ละครั้งอาจย้ายได้น้อยกว่าที่ขอ จึงต้องวนจนครบ
เพิ่มเป็น 10 workers แล้วช้าลง (Traditional) เพราะอะไร	คอขวดอยู่ที่ทรัพยากรร่วม (CPU/หน่วยความจำ/ดิสก์) บนเครื่องเดียว บวกต้นทุน connection, context switch และการเขียนกระจายหลายตำแหน่ง
ถ้า Server รับเกิน 20 connection พร้อมกัน	ที่เกินรอคิวใน thread pool จนมี thread ว่าง ไม่ล่มและไม่ถูกปฏิเสธ
ถ้า worker ตัวหนึ่งพังระหว่างโหลด	`t.get()` โยน `ExecutionException` แล้ว `download` โยน `IOException("worker failed: ...")` การทดสอบหยุดและ pool ถูกปิดใน finally
ผลทดสอบเชื่อถือได้แค่ไหน	ข้อมูลถูกต้องแน่นอน (ตรวจ hash) แต่ตัวเลขเวลาเป็นผลบน localhost ซึ่งมี Page Cache และ loopback จึงเป็นเพียงภาพโดยประมาณของสภาพจริง

12. จับคู่ข้อกำหนดของโจทย์กับตำแหน่งในโค้ด

12.1 ข้อกำหนดของโปรแกรม

ข้อกำหนด	ไฟล์	ตำแหน่งในโค้ด และสิ่งที่ทำ
Client และ Server ทำงานผ่าน TCP ใช้ command-line ได้ ไม่บังคับ GUI	Server.java Client.java	Server: `main` ใช้ `ServerSocketChannel.bind(port)` และ `accept()` (TCP) Client: `new Socket(HOST, port)` ใน `request` / `list` / `fetchTraditional` และ `SocketChannel.open(...)` ใน `fetchNio` ทั้งสองรับค่าจาก `args[]` ของ `main` ไม่มี GUI
Server รองรับหลาย connection พร้อมกัน โดยเลือกใช้ Thread, Thread Pool หรือ Virtual Thread	Server.java	ใช้ **Thread Pool**: `Executors.newFixedThreadPool(20)` ใน `main` วน `while(true)` เรียก `accept()` แล้ว `pool.submit(() -> handle(client))` ให้ 1 connection ต่อ 1 thread ใน pool
Client แบ่งไฟล์เป็น 10 ช่วง ใช้ 10 workers พร้อมกัน แต่ละ worker มี connection, offset และ length ของตนเอง ช่วงข้อมูลต้องไม่ซ้อนกัน	Client.java	`test()` กำหนด `workerCounts = {1, 10}` และเรียก `download(name, workers, ...)` ใน `download()` : `part = size / workers`, `offset = i * part`, ช่วงถัดไปเริ่มต่อจากช่วงก่อนพอดี จึงไม่ซ้อนกัน แต่ละ worker เรียก `fetchRange(...)` ซึ่งเปิด Socket/SocketChannel ใหม่ของตัวเอง

เขียนไฟล์ปลายทางได้ 2 แบบ: part files แล้ว merge หรือเขียนลง final file ตาม offset ด้วย RandomAccessFile/FileChannel (แยก file handle/channel ของแต่ละ worker หรือใช้ positional write อย่างถูกต้อง)	Client.java	เลือกแบบ **เขียนลง final file ตาม offset** (ไม่มี part files จึงไม่ต้อง merge)
		`createFile()` สร้างไฟล์เปล่าขนาดเท่าต้นฉบับด้วย `setLength(size)`
		`fetchTraditional()` : แต่ละ worker เปิด `RandomAccessFile` ของตนเอง แล้ว `file.seek(writeAt)`
		`fetchNio()` : แต่ละ worker เปิด `FileChannel` ของตนเอง แล้ว `file.transferFrom(socket, writeAt + done, ...)` (positional write)

12.2 แนวทางการทำงาน (ทีละขั้น)

ขั้นตอนในโจทย์	ไฟล์	ตำแหน่งในโค้ด และสิ่งที่ทำ
1. Client ขอรายชื่อ/ข้อมูลไฟล์และขนาดไฟล์จาก Server	Client.java Server.java	รายชื่อไฟล์: Client `list()` ส่ง `LIST` ↔ Server `doList()` ขนาดไฟล์: Client `getSize()` ส่ง `INFO` ↔ Server `doInfo()` (`getSize` ถูกเรียกใน `test()` และ `download()`)

2. Client คำนวณ byte range สำหรับ 10 workers โดย worker สุดท้ายรับช่วงที่เหลือ	Client.java	`download()` ในลูป `for (int i = 0; i < workers; i++)`
		`long offset = i * part;`
		`long length = (i == workers - 1) ? size - offset : part;` (คนสุดท้ายรับเศษที่เหลือ)

3. แต่ละ worker เปิด connection ของตนเอง ส่ง GET พร้อม offset/length และรับข้อมูลเฉพาะช่วงนั้น	Client.java	`fetchRange()` ประกอบคำสั่ง `"GET " + name + " " + offset + " " + length + " " + mode` แล้วเรียก `fetchTraditional()` หรือ `fetchNio()` ซึ่งเปิด connection ใหม่ ส่งคำสั่ง อ่าน `OK n` แล้วรับข้อมูลเพียง `length` byte

4a. Server handler อ่านเฉพาะช่วงที่ร้องขอ	Server.java	`handle()` → `doGet()` ตรวจรูปแบบ, ไฟล์ และช่วง (400/404/416)
		`sendTraditional()`: `raf.seek(offset)` แล้วอ่านไม่เกิน `length`
		`sendNio()`: `transferTo(offset + sent, length - sent, client)`

4b. เมื่อครบทุก worker ให้ merge part files หรือยืนยันตำแหน่งที่เขียน	Client.java	ไม่ต้อง merge เพราะเขียนตาม offset ตรง ๆ `download()` วน `t.get()` รอทุก worker เสร็จ (ถ้าตัวใดพังจะโยน `worker failed: ...`)
4c. ตรวจ size/hash	Client.java Server.java	Client `download()`: `Files.size(out) != size` ตรวจขนาด และ `sha256(out).equals(expectedHash)` ตรวจ hash `expectedHash` ได้จาก `getHash()` ส่ง `HASH` ↔ Server `doHash()` และ `sha256()`

สรุปสั้น ๆ สำหรับตอบอาจารย์: Server ใช้ Thread Pool 20 ตัวรับหลาย connection · Client แบ่งไฟล์เป็นช่วงใน `download()` แล้วให้ worker แต่ละตัวเปิด connection และไฟล์ handle ของตนเอง · เขียนลง final file ตาม offset จึงไม่ต้อง merge · ตรวจ size และ SHA-256 ท้ายสุดก่อนนับว่าสำเร็จ

ข้อสังเกตเรื่องจำนวน worker: โจทย์ระบุ 10 workers แต่โค้ดทดสอบทั้ง 1 และ 10 workers (`workerCounts = {1, 10}`) เพื่อเทียบผล ซึ่งตรงกับข้อความใน HOW TO RUN ว่า "อย่างน้อย 1 worker และ 10 workers สำหรับแต่ละวิธี"